学　号：2022212357　　密　级：公开

合肥工業大學

Hefei University of Technology

本科毕业设计（论文）

UNDERGRADUATE THESIS

类　　型：设计

题　　目：可证安全图像隐写系统设计与实现

专业名称：信息安全

入校年份：2022 级

学生姓名：支雅安

指导教师：王垚飞　副教授

学院名称：计算机与信息学院

完成时间：2026 年 4 月

合　肥　工　业　大　学

本科毕业设计（论文）

可证安全图像隐写系统设计与实现

学生姓名：支雅安

学生学号：2022212357

指导教师：王垚飞　副教授

专业名称：信息安全

学院名称：计算机与信息学院

2026 年 4 月

A Dissertation Submitted for the Degree of Bachelor

Design and Implementation of a Provably Secure Image Steganography System

By

ZHI Ya'an

Hefei University of Technology

Hefei, Anhui, P.R.China

April, 2026

毕业设计（论文）独创性声明

本人郑重声明：所呈交的毕业设计（论文）是本人在指导教师指导下进行独立研究工作所取得的成果。据我所知，除了文中特别加以标注和致谢的内容外，设计（论文）中不包含其他人已经发表或撰写过的研究成果，也不包含为获得 合肥工业大学 或其他教育机构的学位或证书而使用过的材料。对本文成果做出贡献的个人和集体，本人已在设计（论文）中作了明确的说明，并表示谢意。

毕业设计（论文）中表达的观点纯属作者本人观点，与合肥工业大学无关。

毕业设计（论文）作者签名：学生签　　签名日期：2026 年 4 月 10 日

毕业设计（论文）版权使用授权书

本学位论文作者完全了解 合肥工业大学 有关保留、使用毕业设计（论文）的规定，即：除保密期内的涉密设计（论文）外，学校有权保存并向国家有关部门或机构送交设计（论文）的复印件和电子光盘，允许设计（论文）被查阅或借阅。本人授权 合肥工业大学 可以将本毕业设计（论文）的全部或部分内容编入有关数据库，允许采用影印、缩印或扫描等复制手段保存、汇编毕业设计（论文）。

（保密的毕业设计（论文）在解密后适用本授权书）

学位论文作者签名：学生签　　指导教师签名：导师签

签名日期：2026 年 4 月 10 日　　签名日期：2026 年 4 月 10 日

摘　要

随着网络通信技术的快速发展，信息安全与隐私保护日益受到重视。图像隐写术作为信息隐藏技术的重要分支，通过将秘密信息嵌入载体图像中实现隐蔽通信，在国家安全、版权保护和隐私通信等领域具有重要应用价值。传统隐写方法在安全性方面存在不足，容易被基于深度学习的隐写分析工具检测。近年来，基于生成模型的可证安全隐写方案为解决这一问题提供了理论保障。

本文设计并实现了一个基于 Pulsar 算法的可证安全图像隐写系统。Pulsar 算法利用扩散模型的逆向采样过程，结合算术编码与伪随机数生成器，将秘密消息编码为模型生成图像的采样路径，从而实现可证安全的信息嵌入，使含密图像与正常生成图像在统计分布上不可区分。

系统采用客户端–服务端架构，服务端基于 Python FastAPI 框架构建，集成 Pulsar 隐写算法，提供 RESTful API 和 WebSocket 实时通信服务；客户端包括基于 Kotlin Jetpack Compose 的 Android 应用和基于 Vue.js 3 的 Web 应用，均实现了用户注册登录、好友管理、实时聊天以及隐写消息收发等完整功能。系统支持普通消息与隐写消息两种通信模式，用户可在聊天过程中无缝切换，实现隐蔽的端到端信息传输。

测试结果表明，系统功能完整、运行稳定，能够正确实现消息的隐写嵌入与提取，跨平台通信兼容性良好。本系统将可证安全隐写理论与实际工程应用相结合，为安全隐蔽通信提供了一种可行的解决方案。

关键词：图像隐写；可证安全；扩散模型；跨平台通信；隐写系统

ABSTRACT

With the rapid development of network communication technologies, information security and privacy protection have attracted increasing attention. Image steganography, as an important branch of information hiding, achieves covert communication by embedding secret messages into cover images and has significant applications in national security, copyright protection, and private communication. Traditional steganographic methods suffer from security deficiencies and are susceptible to detection by deep learning-based steganalysis tools. In recent years, provably secure steganographic schemes based on generative models have provided theoretical guarantees for addressing this problem.

This thesis designs and implements a provably secure image steganography system based on the Pulsar algorithm. The Pulsar algorithm leverages the reverse sampling process of diffusion models, combined with arithmetic coding and pseudo-random number generators, to encode secret messages as sampling paths of model-generated images, thereby achieving provably secure information embedding where stego images are statistically indistinguishable from normally generated images.

The system adopts a client-server architecture. The server is built on the Python FastAPI framework, integrating the Pulsar steganographic algorithm and providing RESTful APIs and WebSocket real-time communication services. The client side includes an Android application based on Kotlin Jetpack Compose and a Web application based on Vue.js 3, both implementing complete functionalities including user registration and login, friend management, real-time chat, and steganographic message transmission. The system supports both normal and steganographic communication modes, allowing users to seamlessly switch during conversations for covert end-to-end information transmission.

Testing results demonstrate that the system is functionally complete and operates stably, correctly performing steganographic embedding and extraction of messages with good cross-platform communication compatibility. This system combines provably secure steganography theory with practical engineering applications, providing a feasible solution for secure covert communication.

KEYWORDS: Image Steganography; Provable Security; Diffusion Model; Cross-platform Communication; Steganography System

目 录

1 绪论 1
1.1 研究背景与意义 1
1.2 国内外研究现状 2
1.2.1 传统图像隐写方法 2
1.2.2 基于深度学习的隐写分析 2
1.2.3 可证安全隐写 3
1.3 主要研究内容 3
1.4 论文组织结构 4
2 相关技术与理论基础 5
2.1 图像隐写术概述 5
2.1.1 空间域隐写 5
2.1.2 变换域隐写 5
2.1.3 基于生成模型的隐写 5
2.2 可证安全隐写理论 6
2.2.1 信息论安全模型 6
2.2.2 Cachin 安全准则 6
2.3 Pulsar 算法原理 6
2.3.1 扩散模型基础 7
2.3.2 Pulsar 隐写编码原理 7
2.3.3 Pulsar 消息提取 8
2.3.4 纠错编码机制 8
2.3.5 安全性分析 8
2.4 关键技术栈 8
2.4.1 FastAPI 框架 8

2.4.2 WebSocket 协议 9
2.4.3 Jetpack Compose 9
2.4.4 Vue.js 3 9
2.4.5 本地存储技术 9
2.5 本章小结 10
3 系统设计 11
3.1 需求分析 11
3.1.1 功能需求 11
3.1.2 非功能需求 11
3.2 系统总体架构 11
3.3 服务端设计 12
3.3.1 API 接口设计 12
3.3.2 数据库设计 12
3.3.3 WebSocket 通信协议设计 14
3.3.4 离线消息队列设计 14
3.3.5 隐写服务集成设计 14
3.4 客户端设计 14
3.4.1 Android 客户端架构 14
3.4.2 Web 客户端架构 15
3.4.3 本地存储设计 15
3.5 安全设计 16
3.5.1 JWT 认证机制 16
3.5.2 密码安全 16
3.5.3 隐写密钥派生 16
3.5.4 单端登录策略 16
3.6 本章小结 17
4 系统实现 18

4.1 服务端实现 18
4.1.1 FastAPI 应用结构 18
4.1.2 Pulsar 算法集成 18
4.1.3 WebSocket 连接管理 19
4.1.4 离线消息队列 19
4.2 Android 客户端实现 19
4.2.1 界面实现 19
4.2.2 Room 数据库实现 20
4.2.3 WebSocket 客户端实现 20
4.2.4 隐写功能实现 20
4.3 Web 客户端实现 21
4.3.1 Vue 组件结构 21
4.3.2 Pinia 状态管理 21
4.3.3 IndexedDB 本地存储 21
4.3.4 WebSocket 与隐写功能 22
4.4 关键功能实现细节 22
4.4.1 单端登录机制 22
4.4.2 好友请求流程 22
4.4.3 隐写消息收发流程 23
4.5 本章小结 23
5 系统测试与分析 24
5.1 测试环境 24
5.2 功能测试 24
5.2.1 用户注册与登录测试 24
5.2.2 好友管理测试 25
5.2.3 消息收发测试 25
5.2.4 隐写嵌入与提取测试 25

5.3 隐写性能分析 26
5.3.1 嵌入容量 26
5.3.2 安全性分析 26
5.3.3 图像质量 26
5.4 跨平台兼容性测试 27
5.5 本章小结 27
6 总结与展望 28
6.1 工作总结 28
6.2 不足与展望 28
参考文献 30
致谢 32
附录 33

插图清单

图 3.1 系统总体架构 12
图 3.2 Android 客户端 MVVM 架构 15

表格清单

表 3.1 服务端 API 接口设计 13
表 3.2 服务端数据库表结构 13
表 3.3 WebSocket 消息类型 14
表 5.1 测试环境配置 24
表 5.2 用户注册与登录测试用例 24
表 5.3 好友管理测试用例 25
表 5.4 消息收发测试用例 25
表 5.5 隐写嵌入与提取测试用例 26
表 5.6 跨平台兼容性测试 27

1 绪论

1.1 研究背景与意义

随着互联网技术的飞速发展和数字化进程的深入推进，网络已经成为人们日常生活中不可或缺的信息交流渠道。然而，网络通信的开放性和共享性也带来了严峻的信息安全挑战。政府、企业和个人的敏感信息在传输过程中面临着被窃取、篡改和监控的风险[1]。传统的加密技术虽然能够保护信息内容的机密性，但加密通信行为本身容易引起监控者的注意，从而招致进一步的审查和干预。

信息隐藏技术（Information Hiding）为解决上述问题提供了一种独特的思路。与加密技术不同，信息隐藏并不改变通信形式的外观，而是将秘密信息嵌入到看似正常的载体媒体（如图像、音频、视频等）中，使得通信双方的交互在外部观察者看来与正常通信无异[2]。图像隐写术（Image Steganography）作为信息隐藏技术的重要分支，因其载体图像在网络中传输量大、格式多样、视觉冗余丰富等特点，成为研究最为活跃的方向之一。

早期的图像隐写方法主要基于空间域修改策略，如最低有效位（Least Significant Bit, LSB）替换[3]。这类方法实现简单，但嵌入操作会不可避免地改变载体图像的统计特性，容易被统计隐写分析方法检测[4]。随着深度学习技术的发展，基于卷积神经网络的隐写分析工具（如 SRNet[5]）展现出了极强的检测能力，对传统隐写方法构成了严重威胁。

为了从根本上解决隐写安全性问题，研究者提出了可证安全隐写（Provably Secure Steganography）的理论框架[6]。该理论指出，如果隐写系统生成的含密载体分布与正常载体分布在统计上不可区分（即 KL 散度为零），则任何检测器都无法以优于随机猜测的概率检测出隐写行为。近年来，基于深度生成模型（如变分自编码器 VAE、生成对抗网络 GAN、扩散模型 Diffusion Model）的隐写方案为实现可证安全提供了新的技术路径[7]。

本文的研究正是在上述背景下展开的。本文选择基于扩散模型的 Pulsar 算法[8]作为核心隐写方案，设计并实现一个完整的可证安全图像隐写通信系统。该系统将可证安全隐写理论与实际工程应用相结合，支持多平台（Android、Web）的隐蔽即时通信，具有以下意义：

1. **理论意义**：将可证安全隐写的理论成果应用于实际系统中，验证其在工程环境

下的可行性和有效性。

2. **应用意义**：为用户提供一种安全、便捷的隐蔽通信工具，满足隐私保护和安全通信的实际需求。
3. **技术意义**：探索扩散模型在隐写领域的应用，为后续研究提供参考实现和工程经验。

1.2 国内外研究现状

1.2.1 传统图像隐写方法

传统图像隐写方法可大致分为空间域方法和变换域方法两大类。

空间域隐写方法直接在图像像素值上进行修改。最经典的是 LSB 替换方法，通过将秘密信息的比特替换载体图像像素的最低有效位实现嵌入[3]。为了减小嵌入对图像统计特性的影响，研究者提出了 LSB 匹配[9]、HUGO（Highly Undetectable Stego）[10]、WOW（Wavelet Obtained Weights）[11] 和 S-UNIWARD[12] 等自适应隐写方法，通过最小化嵌入失真函数来优化嵌入位置的选择。

变换域隐写方法在图像的频域或其他变换域中嵌入信息。典型的方法包括基于离散余弦变换（DCT）的 JPEG 域隐写[13]，如 F5 算法、nsF5 算法[14] 和 J-UNIWARD[12] 等。

尽管上述方法在降低可检测性方面取得了显著进展，但由于嵌入操作本质上是对已有载体的修改，不可避免地会引入统计痕迹，无法在理论上保证绝对安全。

1.2.2 基于深度学习的隐写分析

隐写分析（Steganalysis）旨在检测载体中是否隐藏了秘密信息。传统隐写分析方法依赖手工设计的特征提取器，如 SRM（Spatial Rich Model）[4]。随着深度学习的兴起，基于卷积神经网络的隐写分析方法展现了更优异的检测性能。代表性工作包括 Xu-Net[15]、Ye-Net[16] 和 SRNet[5] 等。这些方法能够自动学习隐写特征，对多种隐写方案都具有较高的检测准确率，进一步凸显了传统隐写方法在安全性方面的不足。

1.2.3 可证安全隐写

可证安全隐写的理论基础由 Hopper 等人[6] 奠定。其核心思想是：隐写系统的安全性可以归结为含密载体分布与正常载体分布之间的统计距离（如 KL 散度）。若两者分布完全一致，则隐写系统是完美安全的（Perfectly Secure）。Cachin[17] 进一步从信息论角度形式化了这一安全准则。

实现可证安全的关键在于隐写编码过程不能对载体分布引入任何偏差。基于深度生成模型的隐写方案通过在模型的采样过程中嵌入信息，天然地满足了这一要求。代表性工作包括：

- 基于 GAN 的方案：利用 GAN 的随机输入向量（latent code）嵌入信息[18]。
- 基于 VAE 的方案：在变分自编码器的隐变量采样过程中嵌入信息。
- 基于 Flow 的方案：利用可逆归一化流模型的采样过程嵌入信息[7]。
- 基于扩散模型的方案：利用扩散模型逆向去噪采样过程中的随机性嵌入信息。Pulsar[8] 是该方向的代表性工作，通过算术编码将秘密信息映射为扩散模型每一步去噪采样的伪随机数序列，实现了高容量、高质量的可证安全隐写。

与其他生成模型相比，扩散模型生成的图像质量更高、多样性更好，是当前最有前景的可证安全隐写载体。

1.3 主要研究内容

本文围绕可证安全图像隐写系统的设计与实现，开展了以下主要工作：

1. **系统架构设计**：设计了基于客户端–服务端的跨平台隐写通信系统架构，服务端负责隐写算法计算和消息路由，客户端（Android 和 Web）负责用户交互和本地数据管理。
2. **Pulsar 算法集成**：将基于扩散模型的 Pulsar 可证安全隐写算法集成到服务端，实现了消息嵌入和提取的 API 接口，支持容量查询和密钥管理。
3. **实时通信系统**：基于 WebSocket 协议实现了服务端与客户端之间的实时双向通信，支持在线状态感知、消息即时推送和离线消息缓存。
4. **用户体系与好友管理**：实现了完整的用户注册登录系统（JWT 认证）、邀请码机制的好友添加功能，以及单端登录安全策略。
5. **多平台客户端开发**：分别使用 Kotlin Jetpack Compose 和 Vue.js 3 开发了功能完

整的 Android 和 Web 客户端，实现了普通消息与隐写消息的无缝切换。

6. **系统测试与验证**：对系统各项功能进行了全面测试，验证了系统的正确性、稳定性和跨平台兼容性。

1.4 论文组织结构

本文共分为六章，各章内容安排如下：

第一章绪论：介绍了研究背景与意义，综述了国内外图像隐写术的研究现状，阐述了本文的主要研究内容和论文组织结构。

第二章相关技术与理论基础：介绍了图像隐写术的基本概念、可证安全隐写的理论基础、Pulsar 算法的工作原理，以及系统开发所涉及的关键技术栈。

第三章系统设计：从需求分析出发，详细阐述了系统的总体架构设计、服务端设计、客户端设计和安全设计方案。

第四章系统实现：分别介绍了服务端、Android 客户端和 Web 客户端的具体实现过程，以及关键功能的实现细节。

第五章系统测试与分析：描述了系统的测试环境、功能测试、隐写性能分析和跨平台兼容性测试的过程与结果。

第六章总结与展望：总结了本文的主要工作和贡献，分析了系统的不足之处，并对未来的研究方向进行了展望。

2 相关技术与理论基础

本章介绍与系统设计和实现密切相关的理论基础和技术栈，包括图像隐写术的基本概念、可证安全隐写理论、Pulsar 算法的工作原理以及系统开发涉及的关键技术。

2.1 图像隐写术概述

图像隐写术是指将秘密信息嵌入到数字图像中，使得含密图像（Stego Image）在视觉上与原始载体图像（Cover Image）无显著差异，从而实现隐蔽通信的技术[1]。根据嵌入域的不同，图像隐写方法主要分为以下几类：

2.1.1 空间域隐写

空间域隐写直接修改图像像素值来嵌入信息。最经典的是 LSB（Least Significant Bit）替换方法，将秘密信息的每一比特替换载体图像对应像素的最低有效位[3]。该方法实现简单、嵌入容量大，但由于直接改变了像素的统计分布，容易被隐写分析检测。

为降低嵌入失真，研究者提出了多种自适应隐写方法。HUGO[10] 通过最小化高阶残差统计量的变化来选择嵌入位置；WOW[11] 利用小波变换获取像素权重，优先在纹理复杂区域嵌入；S-UNIWARD[12] 在空间域和 JPEG 域统一使用方向滤波器组计算嵌入代价，是目前抗检测能力较强的方法之一。

2.1.2 变换域隐写

变换域隐写在图像经过某种数学变换（如离散余弦变换 DCT、离散小波变换 DWT）后的系数上嵌入信息。典型代表是 JPEG 域隐写，如 F5 算法[13] 通过减小 DCT 系数的绝对值嵌入信息，nsF5[14] 进一步避免了收缩问题，J-UNIWARD[12] 则实现了最优的嵌入代价分配。

2.1.3 基于生成模型的隐写

与修改已有图像不同，基于生成模型的隐写方法直接在图像生成过程中嵌入信息，不需要已有的载体图像。这类方法利用深度生成模型（如 GAN、VAE、Flow、扩

散模型）采样过程中的随机性嵌入秘密信息，使得含密图像与模型正常输出的图像在分布上完全一致，从根本上消除了隐写痕迹[7]。

2.2 可证安全隐写理论

2.2.1 信息论安全模型

可证安全隐写（Provably Secure Steganography）的理论基础源于 Cachin[17] 和 Hopper 等人[6] 的工作。其核心思想可以用信息论的语言形式化描述：

设 P_C 表示正常载体（Cover）的分布，P_S 表示含密载体（Stego）的分布。隐写系统的安全性通过两个分布之间的统计距离来度量。常用的度量指标为 KL 散度（Kullback-Leibler Divergence）：

$$D_{KL}(P_S \| P_C) = \sum_x P_S(x) \log \frac{P_S(x)}{P_C(x)} \tag{2.1}$$

2.2.2 Cachin 安全准则

Cachin[17] 提出了基于 KL 散度的隐写安全性分级：

- **完美安全（ϵ-secure，$\epsilon = 0$）**：若 $D_{KL}(P_S \| P_C) = 0$，即含密载体与正常载体的分布完全一致，则任何检测器都无法以优于随机猜测的概率检测隐写行为。
- **近似安全（ϵ-secure，$\epsilon > 0$）**：若 $D_{KL}(P_S \| P_C) \leq \epsilon$，其中 ϵ 为一个很小的正数，则隐写系统具有近似安全性。

实现完美安全的关键在于：隐写编码过程不能对载体分布引入任何偏差。基于生成模型的隐写方案通过在模型的采样过程中嵌入信息，天然地满足了这一要求——因为含密图像也是由同一个生成模型产生的，其分布与正常生成图像完全一致。

2.3 Pulsar 算法原理

Pulsar[8] 是一种基于扩散模型（Diffusion Model）的可证安全图像隐写算法，通过将秘密信息编码为扩散模型逆向采样过程中的伪随机噪声序列，实现高容量、高质量的隐写嵌入。

2.3.1 扩散模型基础

扩散模型[19] 是一类通过逐步去噪过程生成数据的深度生成模型，主要包含前向扩散过程和逆向去噪过程：

前向扩散过程：从真实数据 x_0 出发，逐步添加高斯噪声，经过 T 步后得到近似纯噪声 $x_T \sim \mathcal{N}(0, I)$：

$$q(x_t|x_{t-1}) = \mathcal{N}(x_t; \sqrt{1-\beta_t}x_{t-1}, \beta_t I) \tag{2.2}$$

其中 β_t 为预定义的噪声调度参数。

逆向去噪过程：训练一个神经网络 ϵ_θ 预测每一步的噪声，从纯噪声 x_T 开始逐步去噪恢复图像：

$$p_\theta(x_{t-1}|x_t) = \mathcal{N}(x_{t-1}; \mu_\theta(x_t, t), \sigma_t^2 I) \tag{2.3}$$

其中 μ_θ 由模型预测的噪声计算得出，σ_t 为方差系数。每步去噪都需要引入随机噪声 $z_t \sim \mathcal{N}(0, I)$。

2.3.2 Pulsar 隐写编码原理

Pulsar 算法的核心思想是：利用逆向去噪过程中每一步所需的随机噪声 z_t 作为信息嵌入的载体。具体而言，Pulsar 使用伪随机数生成器（PRG）和算术编码的组合，将秘密消息编码为控制去噪采样的噪声向量。

Pulsar 的嵌入过程主要分为以下步骤：

1. **密钥初始化**：使用共享密钥 K 初始化伪随机数生成器，生成确定性的初始噪声 x_T 和每步的基础噪声。
2. **区域估计**：在编码步执行两次前向计算——分别使用” 全 0” 噪声和” 全 1” 噪声生成两幅参考图像，根据两幅图像像素差异的大小将图像划分为不同的编码区域，并为每个区域分配合适的纠错码参数。
3. **消息编码**：使用 Sage 纠错码对秘密消息进行编码，生成编码后的比特序列。将编码比特映射到对应区域的像素位置，控制该位置使用” 全 0” 还是” 全 1” 噪声向量。

4. **图像生成**：使用编码后的噪声向量完成扩散模型的逆向采样过程，生成含密图像。

2.3.3 Pulsar 消息提取

提取过程是嵌入的逆操作：

1. **密钥恢复**：接收方使用相同的共享密钥 K 初始化伪随机数生成器，恢复与发送方相同的初始噪声和区域划分。
2. **参考图像生成**：分别使用”全 0”和”全 1”噪声生成两幅参考图像。
3. **比特恢复**：对于含密图像的每个编码像素，计算其与两幅参考图像对应像素的距离，取距离更近的一方对应的比特值作为恢复值。
4. **消息解码**：使用 Sage 纠错码对恢复的比特序列进行解码，还原秘密消息。

2.3.4 纠错编码机制

由于图像在存储和传输过程中可能引入精度损失，Pulsar 采用基于 Sage 数学系统的纠错码来增强鲁棒性。系统根据不同区域的误码率自动选择合适的纠错码参数，使用 Reed-Solomon 码作为外码、Reed-Muller 码或 BCH 码作为内码的级联编码方案，在不同信道条件下实现可靠的消息传输。

2.3.5 安全性分析

Pulsar 的可证安全性基于以下事实：含密图像与正常图像都是由同一扩散模型生成的，差异仅在于采样过程中使用的随机噪声来源不同。由于正常生成使用的是真随机数，而隐写生成使用的是密码学安全的伪随机数（PRG），在不知道密钥的情况下，两者生成的噪声序列在计算上不可区分。因此，含密图像与正常图像在统计分布上也不可区分，满足 Cachin 安全准则中的完美安全条件。

2.4 关键技术栈

2.4.1 FastAPI 框架

FastAPI[20] 是一个基于 Python 的现代 Web 框架，具有以下特点：

- 基于 ASGI（Asynchronous Server Gateway Interface）标准，原生支持异步编程。

- 自动生成 OpenAPI 文档和交互式 API 测试界面。
- 基于 Python 类型注解的请求参数验证和序列化。
- 高性能，与 Node.js 和 Go 等框架性能相当。

本系统使用 FastAPI 构建服务端 RESTful API 和 WebSocket 服务。

2.4.2 WebSocket 协议

WebSocket 是一种在单个 TCP 连接上进行全双工通信的协议[21]。与传统的 HTTP 请求-响应模式不同，WebSocket 允许服务端主动向客户端推送消息，适用于实时通信场景。本系统使用 WebSocket 实现即时消息推送、在线状态感知和好友请求通知等功能。

2.4.3 Jetpack Compose

Jetpack Compose[22] 是 Google 推出的 Android 现代声明式 UI 框架。与传统的 XML 布局相比，Compose 直接使用 Kotlin 代码描述 UI，支持响应式编程模型，代码更简洁，开发效率更高。本系统的 Android 客户端完全基于 Jetpack Compose 构建。

2.4.4 Vue.js 3

Vue.js 3[23] 是一个渐进式 JavaScript 前端框架，采用组合式 API（Composition API）提供更灵活的逻辑复用能力。本系统使用 Vue.js 3 配合 TypeScript 开发 Web 客户端，使用 Pinia 进行状态管理，使用 Vue Router 管理页面路由。

2.4.5 本地存储技术

为实现离线数据持久化，系统在不同平台采用了相应的本地存储方案：

- **SQLite**：服务端使用 SQLite 作为轻量级关系型数据库，通过 aiosqlite 库实现异步操作。
- **Room**：Android 客户端使用 Room 持久化库，它是 SQLite 的抽象层，提供编译时 SQL 验证和与 Kotlin 协程的无缝集成。
- **IndexedDB**：Web 客户端使用 IndexedDB 浏览器端数据库存储聊天记录和用户数据，通过 Dexie.js 库简化操作。

2.5 本章小结

本章介绍了图像隐写术的基本概念和分类，阐述了可证安全隐写的理论基础和 Cachin 安全准则，详细分析了 Pulsar 算法的嵌入、提取和纠错编码机制，并介绍了系统开发涉及的关键技术栈。这些理论和技术为后续的系统设计与实现奠定了基础。

3 系统设计

本章从需求分析出发，详细阐述系统的总体架构设计、服务端设计、客户端设计和安全设计方案。

3.1 需求分析

3.1.1 功能需求

通过对可证安全图像隐写通信系统的应用场景分析，确定系统的功能需求如下：

1. **用户管理**：支持用户注册、登录、注销，每个用户拥有唯一的邀请码标识。
2. **好友管理**：通过邀请码添加好友，支持好友请求的发送、接受和拒绝。
3. **实时聊天**：支持一对一文本消息的即时收发，消息持久化存储于本地。
4. **隐写消息**：在聊天过程中支持隐写模式，可将文本消息嵌入图像中发送，接收方可提取和查看隐藏的消息。
5. **独立隐写工具**：提供独立的消息嵌入和提取页面，支持用户手动输入密钥进行隐写操作。
6. **跨平台支持**：系统需同时支持 Android 和 Web 两个平台，不同平台的用户可以互相通信。

3.1.2 非功能需求

1. **安全性**：隐写算法须满足可证安全条件；用户认证采用 JWT 令牌机制；密码存储使用安全的哈希算法。
2. **实时性**：消息收发延迟应在秒级以内（不含隐写计算时间）。
3. **可靠性**：当用户离线时，消息应缓存于服务端，待用户上线后自动投递。
4. **易用性**：客户端界面简洁直观，隐写功能与普通聊天无缝集成。

3.2 系统总体架构

系统采用客户端–服务端（C/S）架构，由服务端、Android 客户端和 Web 客户端三部分组成。系统总体架构如图 3.1所示。

各组件的职责分工如下：

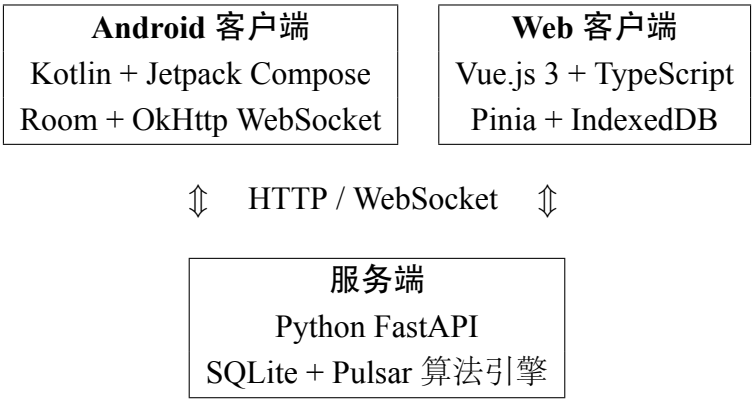

图 3.1　系统总体架构

- **服务端**：负责用户认证与管理、好友关系维护、WebSocket 连接管理与消息路由、离线消息缓存、Pulsar 隐写算法的执行（嵌入与提取计算）。
- **Android 客户端**：提供移动端的用户界面，包括登录注册、联系人管理、聊天界面和隐写工具，使用 Room 数据库本地存储聊天记录。
- **Web 客户端**：提供浏览器端的用户界面，功能与 Android 客户端对齐，使用 IndexedDB 本地存储聊天记录。

服务端与客户端之间通过两种通信方式交互：

- **RESTful API（HTTP）**：用于用户注册登录、隐写嵌入提取、邀请码查询等请求-响应式操作。
- **WebSocket**：用于实时消息推送、好友请求通知、在线状态变化等需要双向实时通信的场景。

3.3 服务端设计

3.3.1 API 接口设计

服务端 API 按照功能模块划分为四组路由，如表 3.1所示。

3.3.2 数据库设计

服务端使用 SQLite 数据库存储用户数据。数据库包含三张核心表，如表 3.2所示。

好友关系不存储在服务端数据库中，而是通过客户端本地数据库维护，服务端仅负责转发好友请求消息。这种设计减轻了服务端的存储负担，同时保护了用户的

表 3.1　服务端 API 接口设计

端点	方法	功能
/api/v1/auth/register	POST	用户注册
/api/v1/auth/login	POST	用户登录
/api/v1/auth/me	GET	获取当前用户信息
/api/v1/invite/my-code	GET	获取自身邀请码
/api/v1/invite/reset	POST	重置邀请码
/api/v1/invite/lookup/{code}	GET	根据邀请码查找用户
/api/v1/invite/user-code/{user_id}	GET	获取指定用户邀请码
/api/v1/stego/embed	POST	消息嵌入
/api/v1/stego/extract	POST	消息提取
/api/v1/stego/capacity	POST	容量检查
/api/v1/stego/max-capacity	GET	获取最大容量
/api/v1/stego/models	GET	获取可用模型列表
/ws?token={jwt}	WebSocket	实时通信连接

表 3.2　服务端数据库表结构

表名	字段	类型	说明
users	id	TEXT (PK)	用户 UUID
	username	TEXT (UNIQUE)	用户名
	password_hash	TEXT	密码哈希
	created_at	DATETIME	创建时间
invite_codes	code	TEXT (PK)	邀请码
	user_id	TEXT (FK, UNIQUE)	所属用户
	created_at	DATETIME	创建时间
message_queue	id	TEXT (PK)	消息 UUID
	to_user_id	TEXT (FK)	目标用户
	payload	TEXT	消息 JSON 内容
	created_at	DATETIME	创建时间
	expires_at	DATETIME	过期时间

社交关系隐私。

3.3.3 WebSocket 通信协议设计

WebSocket 连接建立后，客户端和服务端通过 JSON 格式的消息进行通信。消息类型设计如表 3.3所示。

表 3.3　WebSocket 消息类型

消息类型	说明
chat	普通文本聊天消息
stego	隐写图像消息（携带 base64 图像数据）
friend_request	好友请求
friend_accept	接受好友请求
friend_reject	拒绝好友请求
first_contact	首次联系通知（自动触发）
kicked	单端登录踢出通知

3.3.4 离线消息队列设计

当目标用户不在线时，服务端将消息暂存于 `message_queue` 表中，并设置过期时间（默认 7 天）。当用户重新连接 WebSocket 时，服务端自动查询并投递该用户的所有未过期离线消息，投递完成后从队列中删除。同时，服务端启动定时任务每 5 分钟清理过期消息。

3.3.5 隐写服务集成设计

隐写服务通过 `PulsarService` 类封装 Pulsar 算法，采用单例模式管理模型实例。每个（模型、密钥）组合对应一个独立的 Pulsar 实例，实例创建时自动完成模型加载和区域估计。嵌入和提取操作通过服务类的类方法调用，支持多种预训练模型（CelebA-HQ、Church、Bedroom、Cat）。

3.4 客户端设计

3.4.1 Android 客户端架构

Android 客户端采用 MVVM（Model-View-ViewModel）架构模式，如图 3.2所示。

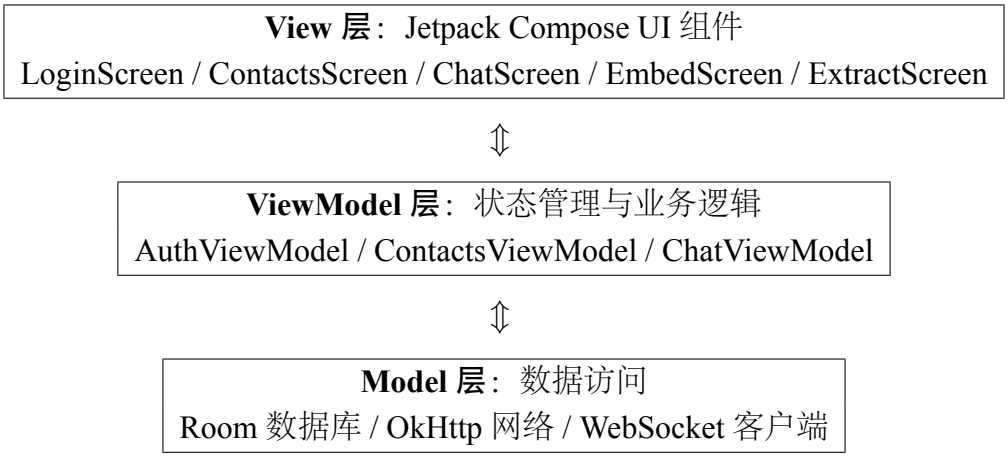

图 3.2　Android 客户端 MVVM 架构

- **View 层**：使用 Jetpack Compose 编写声明式 UI，包括登录注册、联系人列表、聊天界面、隐写嵌入和提取等页面。
- **ViewModel 层**：管理 UI 状态，处理业务逻辑，通过 StateFlow 向 View 层提供响应式数据。
- **Model 层**：包括 Room 数据库（本地存储聊天记录和联系人信息）、OkHttp 网络请求、WebSocket 客户端。

3.4.2 Web 客户端架构

Web 客户端采用 Vue.js 3 组合式 API 开发，使用 Pinia 进行集中式状态管理。

- **视图层**：Vue 单文件组件（SFC），包括 LoginView、ContactsView、ChatView、EmbedView、ExtractView 等。
- **状态管理层**：Pinia Store 管理全局状态，包括 AuthStore（认证状态）、ChatStore（聊天与 WebSocket 连接）、ContactStore（联系人管理）。
- **数据持久层**：使用 IndexedDB（通过 Dexie.js 封装）存储聊天消息和联系人数据。

3.4.3 本地存储设计

Android 客户端的 Room 数据库包含以下实体：

- **Contact**：联系人信息（用户 ID、用户名、邀请码、添加时间）。
- **ChatMessage**：聊天消息（消息 ID、发送者、接收者、内容、时间、消息类型、隐写图像数据）。

- **PendingRequest**：待处理的好友请求。

Web 客户端的 IndexedDB 数据库结构与之对应，使用 Dexie.js 定义 contacts 和 messages 两个对象存储。

3.5 安全设计

3.5.1 JWT 认证机制

系统使用 JSON Web Token（JWT）进行用户身份认证。用户登录成功后，服务端签发包含用户 ID 和用户名的 JWT 令牌，有效期为 7 天。客户端在后续的 HTTP 请求中通过 Authorization 头部携带令牌，WebSocket 连接通过 URL 查询参数传递令牌。服务端使用 HMAC-SHA256 算法验证令牌的完整性和有效性。

3.5.2 密码安全

用户密码使用 PBKDF2-HMAC-SHA256 算法进行哈希存储，迭代次数为 100,000 次，每个用户使用随机生成的 UUID 作为盐值。这种方案有效防止了彩虹表攻击和暴力破解。

3.5.3 隐写密钥派生

在隐写聊天模式下，系统自动使用通信双方的邀请码拼接作为隐写密钥。具体而言，密钥 $K = \text{sender_invite_code} \| \text{receiver_invite_code}$，其中 $\|$ 表示字符串拼接。这种设计保证了每一对通信用户拥有唯一的密钥，且双方都能独立计算出相同的密钥。

3.5.4 单端登录策略

为防止多设备同时登录引发的数据一致性问题，系统实施单端登录策略。当用户在新设备登录时，服务端向旧连接发送 `kicked` 消息通知其被踢出，旧客户端收到通知后清除本地令牌并跳转到登录页面。5 秒后服务端强制关闭旧连接作为兜底措施。

3.6 本章小结

本章从功能需求和非功能需求出发，设计了系统的总体架构，详细阐述了服务端的 API 接口、数据库、WebSocket 通信协议、离线消息队列和隐写服务的设计方案，以及客户端的架构和本地存储设计。同时，从 JWT 认证、密码安全、隐写密钥派生和单端登录四个方面介绍了系统的安全设计。

4 系统实现

本章分别介绍服务端、Android 客户端和 Web 客户端的具体实现过程，以及关键功能的实现细节。

4.1 服务端实现

4.1.1 FastAPI 应用结构

服务端基于 FastAPI 框架构建，采用模块化的项目结构。主要目录组织如下：

```
server/
    main.py              # 应用入口
    config.py            # 配置项
    database.py          # 数据库初始化
    dependencies.py      # 依赖注入
    api/v1/
        auth.py          # 认证路由
        invite.py        # 邀请码路由
        stego.py         # 隐写路由
        ws.py            # WebSocket 路由
    services/
        auth_service.py  # 认证服务
        pulsar_service.py# Pulsar 服务封装
        ws_manager.py    # WebSocket 连接管理
        queue_service.py # 离线消息队列
    models/              # 数据模型
```

应用启动时通过 FastAPI 的生命周期管理（lifespan）完成数据库初始化和定时任务注册。路由通过 `include_router` 方法注册到主应用实例。CORS 中间件允许跨域请求以支持 Web 客户端访问。

4.1.2 Pulsar 算法集成

Pulsar 算法通过 `PulsarService` 类进行封装，该类采用类方法设计，主要包含以下功能：

- **实例管理**：使用字典缓存 Pulsar 实例，以（模型 ID、密钥）组合作为缓存键。首次请求时自动加载预训练模型并执行区域估计，后续请求复用已有实例。
- **嵌入操作**：接收消息字节、模型 ID 和密钥，调用 Pulsar 的 `generate_with_regions` 方法生成含密图像，将图像保存为 16 位 PNG 格式并返回字节数据。

- **提取操作**：接收含密图像字节、模型 ID 和密钥，调用 `reveal_with_regions` 方法提取隐藏消息。
- **容量查询**：返回当前模型和密钥配置下的最大可嵌入消息字节数。

Pulsar 服务通过将算法库路径动态添加到 Python 搜索路径实现导入。算法使用预训练的 DDPM（Denoising Diffusion Probabilistic Model）UNet2D 模型，支持 CUDA、MPS 和 CPU 三种计算后端。

4.1.3 WebSocket 连接管理

`ConnectionManager` 类负责管理所有活跃的 WebSocket 连接，使用字典维护用户 ID 到 WebSocket 实例的映射关系。核心实现包括：

- **连接建立**：验证 JWT 令牌后建立连接。若该用户已有活跃连接（多设备登录），先向旧连接发送 `kicked` 消息，并设置 5 秒延迟强制关闭。
- **消息路由**：根据消息中的目标用户 ID 查找对应的 WebSocket 连接，若目标用户在线则直接转发，否则将消息加入离线队列。
- **离线消息投递**：用户连接建立后，自动查询并投递该用户的所有缓存消息。

4.1.4 离线消息队列

离线消息使用 SQLite 的 `message_queue` 表实现持久化存储。消息入队时设置 7 天过期时间，服务端通过 `asyncio.create_task` 启动定时清理任务，每 5 分钟删除过期消息。用户重新上线时，系统自动投递所有未过期消息并从队列中移除。

4.2 Android 客户端实现

4.2.1 界面实现

Android 客户端使用 Jetpack Compose 构建声明式 UI，主要包含以下页面：

- **LoginScreen**：登录和注册界面，支持用户名密码输入和表单验证。
- **ContactsScreen**：联系人列表，显示所有好友及其在线状态，支持通过邀请码添加好友。
- **ChatScreen**：聊天界面，显示消息列表，支持普通消息和隐写消息的发送与接收。隐写模式下，用户输入的文本会先通过服务端嵌入为图像再发送。

- **EmbedScreen**：独立的消息嵌入工具，支持手动输入密钥和消息进行嵌入。
- **ExtractScreen**：独立的消息提取工具，支持上传含密图像进行提取。

页面导航使用 Jetpack Navigation Compose 实现，底部导航栏提供聊天、隐写工具和设置三个入口。隐写工具页面通过 Material 3 的 `TabRow` 组件实现嵌入和提取页面的切换。

4.2.2 Room 数据库实现

使用 Room 持久化库定义本地数据库，包含以下 DAO（Data Access Object）操作：

- `ContactDao`：联系人的增删改查，支持按用户名搜索。
- `ChatMessageDao`：聊天消息的插入和按联系人 ID 查询，支持按时间排序。
- `PendingRequestDao`：好友请求的插入、查询和删除。

数据库版本管理使用 Room 的自动迁移功能。

4.2.3 WebSocket 客户端实现

Android 端使用 OkHttp 库实现 WebSocket 客户端。在 `ChatViewModel` 中管理连接生命周期：

- 使用 `OkHttpClient` 创建 WebSocket 连接，URL 中携带 JWT 令牌。
- 重写 `onMessage` 回调处理接收到的消息，根据消息类型分发到不同的处理函数(聊天消息、好友请求、隐写消息、踢出通知等)。
- 重写 `onClosing` 回调完成 WebSocket 关闭握手（OkHttp 要求显式调用 `close` 完成握手)。
- 连接断开后支持自动重连机制。

4.2.4 隐写功能实现

聊天界面中的隐写模式通过一个切换开关控制：

1. 用户切换到隐写模式后，发送消息前先调用 `/api/v1/stego/max-capacity` 接口查询当前密钥配置的最大容量，验证消息长度是否在限制范围内。
2. 发送时调用 `/api/v1/stego/embed` 接口将消息嵌入图像，密钥由双方邀请码自动拼接生成。

3. 嵌入成功后，将含密图像的 base64 数据作为 `stego` 类型消息通过 WebSocket 发送给对方。
4. 接收方收到隐写消息后，在聊天界面显示图像，用户可点击提取按钮调用 `/api/v1/stego/extra` 接口查看隐藏的文本消息。

4.3 Web 客户端实现

4.3.1 Vue 组件结构

Web 客户端的主要页面组件包括：

- **LoginView**：登录注册页面，使用响应式表单。
- **ContactsView**：联系人管理，显示好友列表和待处理请求。
- **ChatView**：聊天界面，支持普通和隐写两种消息模式。
- **EmbedView**：消息嵌入工具页面。
- **ExtractView**：消息提取工具页面。
- **BottomNav**：底部导航栏组件。

4.3.2 Pinia 状态管理

使用 Pinia 管理全局状态，定义了以下 Store：

- **AuthStore**：管理用户认证状态，包括令牌存储、登录登出操作。令牌持久化到 `localStorage`。
- **ChatStore**：管理 WebSocket 连接和消息收发。负责建立和维护 WebSocket 连接，处理各类消息的接收和分发，管理在线状态。
- **ContactStore**：管理联系人列表和好友请求，数据持久化到 IndexedDB。

4.3.3 IndexedDB 本地存储

使用 Dexie.js 封装 IndexedDB 操作，定义了 `contacts` 和 `messages` 两个对象存储。消息按联系人 ID 建立索引，支持高效的按会话查询。数据库随用户登录而创建，登出时可选择清除。

4.3.4 WebSocket 与隐写功能

Web 客户端的 WebSocket 和隐写功能实现逻辑与 Android 端基本一致。WebSocket 使用浏览器原生 `WebSocket` API，在 ChatStore 中管理连接生命周期。需注意的响应式处理细节包括：好友请求列表使用展开赋值（spread）触发 Vue 的响应式更新，避免直接 `push` 不触发视图刷新的问题。

隐写图像在 Web 端以 base64 字符串存储（纯 base64，不含 `data:` 前缀），渲染时动态添加 `` 前缀。

4.4 关键功能实现细节

4.4.1 单端登录机制

单端登录涉及服务端和客户端的协同处理：

1. **服务端**：`ConnectionManager.connect` 方法在新连接到来时，检查该用户是否已有活跃连接。若有，先发送 `kicked` 消息，然后创建异步任务在 5 秒后强制关闭旧连接。
2. **Android 客户端**：`ChatViewModel` 监听 `kicked` 消息，收到后断开 WebSocket、通过 SharedFlow 发射事件通知 UI 层显示提示信息并跳转登录页面。
3. **Web 客户端**：ChatStore 监听 `kicked` 消息，断开 WebSocket 后通过 DOM 自定义事件（CustomEvent）通知 `App.vue` 组件执行页面跳转。

主动登出清除所有本地数据（令牌、聊天记录、联系人），而被踢出仅清除令牌，保留聊天记录便于用户重新登录后查看。

4.4.2 好友请求流程

好友添加采用基于邀请码的异步请求机制：

1. 用户 A 输入用户 B 的邀请码，客户端调用 `/api/v1/invite/lookup` 查询对应用户信息。
2. 确认后，客户端通过 WebSocket 向用户 B 发送 `friend_request` 消息。若 B 不在线，消息暂存于离线队列。
3. 用户 B 收到请求后可选择接受或拒绝。接受时发送 `friend_accept` 消息，同时双方在本地数据库中添加对方为联系人。

4. 添加联系人时自动发送 `first_contact` 通知，确保双方联系人列表同步。

需要处理的边界情况包括：重复请求的去重、离线请求的暂存与恢复、接受请求时恢复所有暂存的聊天消息等。

4.4.3 隐写消息收发流程

隐写消息的完整收发流程如下：

1. **发送方**：用户切换到隐写模式，输入文本消息。客户端获取双方邀请码拼接为密钥，调用嵌入 API。API 返回含密图像的 base64 数据后，通过 WebSocket 以 `stego` 类型发送。
2. **服务端**：接收 WebSocket 消息，根据目标用户 ID 转发（或入离线队列）。
3. **接收方**：收到 `stego` 消息后，在聊天界面显示图像预览。用户点击提取按钮，客户端使用相同的密钥调用提取 API，解密还原文本消息并显示。

4.5 本章小结

本章详细介绍了系统三个组件的具体实现过程。服务端通过模块化的 FastAPI 应用集成了 Pulsar 算法、WebSocket 通信和离线消息队列；Android 客户端基于 MVVM 架构使用 Jetpack Compose 构建了完整的 UI 和数据管理层；Web 客户端使用 Vue.js 3 和 Pinia 实现了与 Android 端功能对齐的浏览器应用。同时，本章重点阐述了单端登录、好友请求和隐写消息收发三个关键功能的跨端协同实现细节。

5 系统测试与分析

本章介绍系统的测试环境、功能测试、隐写性能分析以及跨平台兼容性测试的过程与结果。

5.1 测试环境

系统测试所使用的软硬件环境如表 5.1所示。

表 5.1　测试环境配置

项目	配置
操作系统	Ubuntu 22.04 LTS
CPU	Intel Core i7 / AMD Ryzen 7
GPU	NVIDIA GPU（支持 CUDA）
内存	16 GB
Python 版本	3.10+（Sage 环境）
Android 测试设备	Android 12+ 物理设备/模拟器
Web 浏览器	Google Chrome 120+

服务端运行在本地开发环境中，使用 `uvicorn` 作为 ASGI 服务器，监听 8000 端口。Pulsar 算法在 Sage 数学软件环境中运行，使用 CUDA 加速扩散模型的推理计算。

5.2 功能测试

5.2.1 用户注册与登录测试

测试了用户注册和登录的正常流程与异常处理，测试用例及结果如表 5.2所示。

表 5.2　用户注册与登录测试用例

编号	测试内容	结果
T-01	正常注册新用户	通过
T-02	注册已存在的用户名	通过（正确提示错误）
T-03	正确用户名密码登录	通过
T-04	错误密码登录	通过（正确提示错误）
T-05	登录后获取用户信息	通过
T-06	注册后自动生成邀请码	通过

5.2.2 好友管理测试

测试了好友添加的完整流程，包括邀请码查询、好友请求发送与处理等，测试用例及结果如表 5.3所示。

表 5.3　好友管理测试用例

编号	测试内容	结果
T-07	通过邀请码查找用户	通过
T-08	发送好友请求（对方在线）	通过
T-09	发送好友请求（对方离线）	通过（离线缓存生效）
T-10	接受好友请求	通过
T-11	拒绝好友请求	通过
T-12	重复发送好友请求（去重）	通过

5.2.3 消息收发测试

测试了普通消息和隐写消息在不同场景下的收发功能,测试用例及结果如表 5.4所示。

表 5.4　消息收发测试用例

编号	测试内容	结果
T-13	发送普通文本消息（双方在线）	通过
T-14	发送普通文本消息（对方离线后上线）	通过
T-15	发送隐写消息（嵌入与传输）	通过
T-16	接收隐写消息并提取文本	通过
T-17	消息本地持久化存储	通过
T-18	隐写图像保存到本地	通过

5.2.4 隐写嵌入与提取测试

测试了独立隐写工具页面的嵌入与提取功能，测试用例及结果如表 5.5所示。

表 5.5　隐写嵌入与提取测试用例

编号	测试内容	结果
T-19	正常嵌入短文本消息	通过
T-20	嵌入后正确提取消息	通过
T-21	超出容量限制时提示错误	通过
T-22	使用错误密钥提取（应失败）	通过（输出乱码）
T-23	容量查询接口测试	通过

5.3 隐写性能分析

5.3.1 嵌入容量

Pulsar 算法的嵌入容量取决于扩散模型生成图像的编码区域划分结果。在使用 CelebA-HQ 256×256 预训练模型的条件下，系统的典型嵌入容量约为数百字节。具体容量因密钥不同而略有变化，因为密钥决定了伪随机数生成器的状态，进而影响区域划分结果。

5.3.2 安全性分析

Pulsar 算法的可证安全性已在原始论文中得到理论证明[8]。其安全性保证基于以下两点：

1. **分布一致性**：含密图像与正常生成图像都由同一扩散模型生成，差异仅在于采样噪声的来源（PRG vs 真随机数），在密码学安全假设下两者不可区分。
2. **密钥安全性**：不知道密钥的攻击者无法确定编码区域和噪声分配，因此无法从图像中恢复秘密信息。

这意味着对于任何多项式时间的隐写分析检测器，检测含密图像的成功概率不优于随机猜测。

5.3.3 图像质量

Pulsar 生成的图像质量完全取决于底层扩散模型的生成能力。由于信息嵌入仅改变了采样路径而非模型本身，含密图像的视觉质量与正常生成图像无差异。使用 CelebA-HQ 预训练模型时，生成的人脸图像分辨率为 256×256，具有较高的真实感。

5.4 跨平台兼容性测试

系统设计的一个重要目标是支持跨平台通信。测试了 Android 客户端与 Web 客户端之间的互操作性，结果如表 5.6所示。

表 5.6　跨平台兼容性测试

编号	测试场景	结果
T-24	Android 发送普通消息 → Web 接收	通过
T-25	Web 发送普通消息 → Android 接收	通过
T-26	Android 发送隐写消息 → Web 提取	通过
T-27	Web 发送隐写消息 → Android 提取	通过
T-28	Android 发送好友请求 → Web 处理	通过
T-29	Web 发送好友请求 → Android 处理	通过
T-30	单端登录：Android 登录后 Web 被踢出	通过
T-31	单端登录：Web 登录后 Android 被踢出	通过

测试结果表明，Android 客户端和 Web 客户端之间能够正常互相发送普通消息和隐写消息，好友管理和单端登录功能跨平台工作正常。base64 编码的隐写图像在两个平台间传输和解析无兼容性问题。

5.5 本章小结

本章从功能测试、隐写性能分析和跨平台兼容性三个维度对系统进行了全面测试。功能测试覆盖了用户管理、好友管理、消息收发和隐写操作等核心功能，共计 31 个测试用例全部通过。隐写性能分析验证了系统在安全性和图像质量方面的表现。跨平台兼容性测试确认了 Android 和 Web 客户端之间的完整互操作性。测试结果表明系统功能完整、运行稳定。

6 总结与展望

6.1 工作总结

本文围绕可证安全图像隐写系统的设计与实现展开研究，主要完成了以下工作：

1. **系统架构设计**：设计了基于客户端–服务端的跨平台隐写通信系统架构。服务端基于 Python FastAPI 框架构建，负责用户管理、消息路由和隐写计算；客户端分别使用 Kotlin Jetpack Compose（Android）和 Vue.js 3（Web）开发，实现了完整的用户交互和本地数据管理。
2. **Pulsar 算法集成**：将基于扩散模型的 Pulsar 可证安全隐写算法成功集成到服务端，封装了嵌入、提取和容量查询接口。系统支持多种预训练模型，通过区域估计和纠错编码机制保证消息的可靠嵌入与提取。
3. **实时通信系统**：基于 WebSocket 协议实现了全双工实时通信，支持消息即时推送、在线状态感知、好友请求通知和离线消息缓存。设计并实现了单端登录安全策略，防止多设备同时登录导致的数据一致性问题。
4. **用户体系与社交功能**：实现了完整的用户注册登录系统、基于邀请码的好友添加机制，以及好友请求的异步处理流程。用户认证基于 JWT 令牌，密码使用 PBKDF2 安全哈希存储。
5. **隐写通信功能**：在聊天界面中集成了隐写模式，用户可在普通消息和隐写消息之间无缝切换。系统自动使用通信双方的邀请码派生隐写密钥，实现了端到端的隐蔽信息传输。同时提供了独立的隐写工具页面，支持手动嵌入和提取操作。
6. **系统测试验证**：对系统进行了全面的功能测试、隐写性能分析和跨平台兼容性测试，共计 31 个测试用例全部通过，验证了系统的正确性和稳定性。

6.2 不足与展望

尽管本系统已实现了基本完整的可证安全隐写通信功能，但仍存在一些不足之处，有待在后续工作中改进：

1. **嵌入容量有限**：受限于扩散模型的采样步数和图像分辨率，当前系统的单幅图像嵌入容量约为数百字节，仅适合短文本消息的传输。未来可通过支持更大分辨率的模型、优化区域划分算法或采用多图像分块嵌入策略来提升容量。
2. **计算延迟较高**：Pulsar 算法涉及多步扩散模型推理和 Sage 纠错编码计算，单次

嵌入或提取操作耗时较长。未来可通过模型量化、蒸馏或使用更高效的扩散模型采样器（如 DDIM 更少步数）来降低延迟。

3. **服务端集中式计算**：当前隐写计算完全在服务端执行，存在单点瓶颈和隐私风险。未来可探索将部分计算迁移到客户端（如使用 WebAssembly 或移动端 GPU 加速），或采用分布式计算架构分担负载。
4. **群组聊天支持**：当前系统仅支持一对一聊天。未来可扩展为群组聊天功能，但需要解决群组密钥管理和多方隐写密钥协商的问题。
5. **界面美化**：当前客户端界面为功能验证阶段，UI 设计较为简洁。未来可进一步优化视觉设计，提升用户体验。
6. **文件类型扩展**：当前系统仅支持文本消息的隐写传输。未来可扩展支持文件、图片等多种数据类型的隐写嵌入，拓展应用场景。

总体而言，本文设计并实现的可证安全图像隐写系统将理论研究成果与工程实践相结合，验证了可证安全隐写技术在实际通信系统中的可行性，为安全隐蔽通信领域提供了一种有价值的解决方案。

参考文献

[1] FRIDRICH J. Steganography in Digital Media: Principles, Algorithms, and Applications[M]. Cambridge University Press, 2009.

[2] KATZENBEISSER S, PETITCOLAS F A P. Information Hiding: Techniques for Steganography and Digital Watermarking[M]. Artech House, 2000.

[3] CHAN C K, CHENG L M. Hiding Data in Images by Simple LSB Substitution[J]. Pattern Recognition, 2004, 37(3): 469-474.

[4] FRIDRICH J, KODOVSKÝ J. Rich Models for Steganalysis of Digital Images[C]//IEEE Transactions on Information Forensics and Security: vol. 7: 3. 2012: 868-882.

[5] BOROUMAND M, CHEN M, FRIDRICH J. Deep Residual Network for Steganalysis of Digital Images[J]. IEEE Transactions on Information Forensics and Security, 2019, 14(5): 1181-1193.

[6] HOPPER N J, LANGFORD J, VON AHN L. Provably Secure Steganography[J]. IEEE Transactions on Computers, 2009, 58(5): 662-676.

[7] DE WITT C S, SOKOTA S, KOLTER J Z, et al. Perfectly Secure Steganography Using Minimum Entropy Coupling[C]//Proceedings of the International Conference on Machine Learning (ICML). 2023.

[8] CHEN G, TAN S, LI B, et al. Pulsar: Secure Steganography for Diffusion Models[J]. ArXiv preprint arXiv:2403.09800, 2024.

[9] MIELIKAÏNEN J. LSB Matching Revisited[J]. IEEE Signal Processing Letters, 2006, 13(5): 285-287.

[10] PEVNÝ T, FILLER T, BAS P. Using High-Dimensional Image Models to Perform Highly Undetectable Steganography[C]//Proceedings of the 12th International Conference on Information Hiding. 2010: 161-177.

[11] HOLUB V, FRIDRICH J. Designing Steganographic Distortion Using Directional Filters[C]//IEEE International Workshop on Information Forensics and Security (WIFS). 2012: 234-239.

[12] HOLUB V, FRIDRICH J, DENEMARK T. Universal Distortion Function for Steganography in an Arbitrary Domain[J]. EURASIP Journal on Information Security, 2014, 2014(1): 1-13.

[13] WESTFELD A. F5—A Steganographic Algorithm[C]//Proceedings of the 4th International Workshop on Information Hiding. 2001: 289-302.

[14] FRIDRICH J, PEVNÝ T, KODOVSKÝ J. Statistically Undetectable JPEG Steganography: Dead Ends, Challenges, and Opportunities[C]//Proceedings of the 9th ACM Workshop on Multimedia and Security. 2007: 3-14.

[15] XU G, WU H Z, SHI Y Q. Structural Design of Convolutional Neural Networks for Steganalysis[C]//IEEE Signal Processing Letters: vol. 23: 5. 2016: 708-712.

[16] YE J, NI J, YI Y. Deep Learning Hierarchical Representations for Image Steganalysis[C]//IEEE Transactions on Information Forensics and Security: vol. 12: 11. 2017: 2545-2557.

[17] CACHIN C. An Information-Theoretic Model for Steganography[C]//Information and Computation: vol. 192: 1. 2004: 41-56.

[18] HU D, WANG L, JIANG W, et al. A Novel Image Steganography Method via Deep Convolutional Generative Adversarial Networks[C]//: vol. 6. 2018: 38303-38314.

[19] HO J, JAIN A, ABBEEL P. Denoising Diffusion Probabilistic Models[J]. Advances in Neural Information Processing Systems, 2020, 33: 6840-6851.

[20] RAMÍREZ S. FastAPI: Modern, Fast (High-Performance), Web Framework for Building APIs with Python[EB/OL]. 2019. https://fastapi.tiangolo.com/.

[21] FETTE I, MELNIKOV A. The WebSocket Protocol[Z]. RFC 6455. 2011.

[22] Google. Jetpack Compose: Android's Modern Toolkit for Building Native UI[EB/OL]. 2021. https://developer.android.com/jetpack/compose.

[23] YOU E. Vue.js: The Progressive JavaScript Framework[EB/OL]. 2020. https://vuejs.org/.

致谢

这里是致谢。这里是致谢。这里是致谢。这里是致谢。这里是致谢。这里是致谢。这里是致谢。这里是致谢。这里是致谢。这里是致谢。这里是致谢。这里是致谢。

作者：支雅安

2026 年 4 月 10 日

附录

附录 A　服务端核心代码

A.1　Pulsar 服务封装

```
class PulsarService:
    _instances: Dict[str, Any] = {}
    _lock = threading.Lock()

    @classmethod
    def get_instance(cls, model_id: str, key: bytes) -> Any:
        cache_key = f"{model_id}:{key.hex()}"
        with cls._lock:
            if cache_key not in cls._instances:
                repo = MODELS[model_id]["repo"]
                instance = pulsar_module.Pulsar(
                    seed=key, repo=repo, benchmarks=False
                )
                instance.estimate_regions(
                    n_hist_bins=100, n_to_gen=1, end_to_end=True
                )
                cls._instances[cache_key] = instance
            return cls._instances[cache_key]

    @classmethod
    def embed(cls, message: bytes, model_id: str,
              key: bytes) -> bytes:
        instance = cls.get_instance(model_id, key)
        results = instance.generate_with_regions(message)
        last = instance.scheduler.num_inference_steps - 1
        hidden_sample = results["samples"][last]["hidden"]
        with tempfile.NamedTemporaryFile(
            suffix=".png", delete=False
        ) as f:
            temp_path = f.name
        try:
            instance.save_sample(hidden_sample, temp_path)
            with open(temp_path, "rb") as f:
                png_data = f.read()
        finally:
            os.unlink(temp_path)
        return png_data
```

A.2 WebSocket 连接管理

```
class ConnectionManager:
    def __init__(self):
        self.active: dict[str, WebSocket] = {}

    async def connect(self, user_id: str, ws: WebSocket):
        await ws.accept()
        old = self.active.get(user_id)
        if old:
            try:
                await old.send_text(json.dumps({
                    "type": "kicked",
                    "payload": {
                        "reason": "Logged in on another device"
                    }
                }))
            except Exception:
                pass
            async def _force_close():
                await asyncio.sleep(5)
                try:
                    await old.close(code=4001)
                except Exception:
                    pass
            asyncio.create_task(_force_close())
        self.active[user_id] = ws

    async def send_to_user(self, user_id: str,
                           message: dict) -> bool:
        ws = self.active.get(user_id)
        if ws:
            await ws.send_text(json.dumps(message))
            return True
        return False
```

附录 B 伪随机数生成器核心代码

```
class Prg:
    def __init__(self, key, seed):
        self.key = key
        self.val = b"\x01" * 64
        self.reseed(seed)
        self.byte_index = 0
        self.bit_index = 0

    def hmac(self, key, val):
        return hmac.new(
```

```
            key, val, hashlib.sha512
        ).digest()

    def reseed(self, data=b""):
        self.key = self.hmac(
            self.key, self.val + b"\x00" + data
        )
        self.val = self.hmac(self.key, self.val)
        if data:
            self.key = self.hmac(
                self.key, self.val + b"\x01" + data
            )
            self.val = self.hmac(self.key, self.val)

    def get_bits(self, n):
        xs = np.zeros(n, dtype=bool)
        for i in range(0, n):
            xs[i] = (
                self.val[self.byte_index]
                >> (7 - self.bit_index)
            ) & 1
            self.bit_index += 1
            if self.bit_index >= 8:
                self.bit_index = 0
                self.byte_index += 1
            if self.byte_index >= 8:
                self.byte_index = 0
                self.val = self.hmac(self.key, self.val)
        self.reseed()
        return xs
```